



Casino Royale Premium

Sistema de Casino con Interfaz Gráfica Avanzada v1.0

CLIENTE:

Uso personal / Proyecto académico

TIPO:

Aplicación de escritorio — Casino multijuego

DESARROLLADOR:

Luis Villegas Rivera

FECHA:

2026-03-09

TECNOLOGÍAS:

Python · Flet · SQLite · PyInstaller

ÍNDICE DE CONTENIDOS

- 1. INFORMACIÓN GENERAL**
 - 1.1 Objetivo del sistema
 - 1.2 Alcance
- 2. CONCEPTO DEL PROYECTO**
 - 2.1 Título
 - 2.2 Descripción general
 - 2.3 Funcionalidades destacadas
- 3. JUSTIFICACIÓN DEL PROYECTO**
 - 3.1 Necesidad del mercado
 - 3.2 Innovación tecnológica
 - 3.3 Beneficios esperados
- 4. TECNOLOGÍAS EMPLEADAS**
 - 4.1 Lenguaje de programación
 - 4.2 Base de datos
 - 4.3 Librerías principales
 - 4.4 Arquitectura
- 5. INGENIERÍA DEL SOFTWARE**
 - 5.1 Storyboard
 - 5.2 Diagrama de flujo (lógico)
 - 5.3 Estructura del proyecto
- 6. DIFICULTADES ENCONTRADAS**
 - 6.1 Técnicas
 - 6.2 Base de datos
 - 6.3 Lógica de negocio
- 7. ANÁLISIS DAFO**
 - 7.1 Matriz DAFO
- 8. ANÁLISIS DE DATOS**
 - 8.1 Tablas principales
- 9. PROYECCIÓN FUTURA**
 - 9.1 Mejoras inmediatas
 - 9.2 Funcionalidades avanzadas
- 10. CONCLUSIONES**
 - 10.1 Logros alcanzados
 - 10.2 Valoración personal
 - 10.3 Impacto esperado

1. INFORMACIÓN GENERAL

Cliente:	Uso personal / Proyecto académico / Clubes privados
Tipo de proyecto:	Aplicación de escritorio — Casino multijuego con GUI avanzada
Desarrollador:	Luis Villegas Rivera
Fecha de entrega:	2026-03-09
Tecnologías:	Python · Flet · SQLite · PyInstaller · Winsound

1.1 Objetivo del sistema

Desarrollar una aplicación de escritorio premium que simule un casino completo, permitiendo a los usuarios jugar a Slots, Ruleta Europea y Blackjack con una interfaz gráfica de alta calidad construida en Flet. El sistema incluye gestión de saldo personalizado por jugador, autenticación segura con SQLite, historial de movimientos, efectos de sonido y generación de ejecutable portable mediante PyInstaller.

1.2 Alcance

- Autenticación con roles: registro, login y recuperación de contraseña
- Tres juegos completos: Tragaperras 3×3, Ruleta Europea (37 casillas) y Blackjack
- Sistema de caja: depósitos, retiros y saldo persistente por jugador
- Historial completo de jugadas con balance por sesión
- Diseño premium con paleta oscura dorada y animaciones fluidas
- Efectos de sonido integrados (acción, victoria, tick de ruleta)
- Límite de intentos de login con bloqueo temporal por seguridad
- Distribución portable: ejecutable único generado con PyInstaller
- Arquitectura modular por vistas con separación de lógica y presentación

2. CONCEPTO DEL PROYECTO

2.1 Título

Sistema Premium de Casino Multijuego con Interfaz Gráfica de Alta Fidelidad

2.2 Descripción general

Casino Royale Premium es una aplicación de escritorio construida con Python y Flet que recrea la experiencia visual de un casino de alta gama. Incorpora tres juegos completamente funcionales (Slots, Ruleta y Blackjack), un sistema de caja con persistencia SQLite, autenticación por usuario con pregunta de seguridad, y una interfaz visualmente cuidada con paleta oscura, acentos dorados y animaciones tipográficas. El sistema puede distribuirse como ejecutable único gracias a PyInstaller.

2.3 Funcionalidades destacadas

Tragaperras 3×3 — Royal Slots

- Cuadrícula 3 columnas × 3 filas con 6 símbolos gráficos (PNG)
- 5 líneas de pago: 3 horizontales + 2 diagonales
- Sistema de pesos por símbolo (mayor frecuencia para símbolos comunes)
- Animación de rodillo por columnas con intervalos de parada escalonados
- Celdas ganadoras resaltadas con borde dorado y sombra animada
- Tabla de pagos visible con multiplicadores (x2 a x50)

Ruleta Europea — Roulette Royale

- Rueda SVG giratoria con 37 casillas (0–36) en orden europeo oficial
- Tablero completo: plenos, docenas, columnas, rojo/negro, par/impar, 1–18/19–36
- Animación de bola realista con desaceleración progresiva y efecto bounce
- Fichas de 6 denominaciones seleccionables (1–500\$) con bandeja de fichas
- Sistema de deshacer última apuesta y limpiar mesa
- Cálculo de ganancias con multiplicadores 2:1, 3:1 y 36:1

Blackjack — Royale Blackjack

- Baraja de 6 mazos barajada con fuente criptográfica (secrets)
- Cartas gráficas con símbolo central, esquinas y gradiente de fondo
- Lógica completa: Blackjack 3:2, dealer con regla H17, carta oculta inicial
- Detección automática de Blackjack, bust y empate
- Animaciones de entrada de carta con offset y opacidad
- Gestión de saldo, apuesta y resultado en tiempo real

Caja y Gestión de Saldo

- Panel de saldo en tiempo real con actualización tras cada jugada
- Botones de cantidad rápida (\$10, \$50, \$100, \$500, \$1000)
- Historial de últimos 8 movimientos con icono, fecha y delta de saldo

- SnackBar de notificación para operaciones exitosas o errores

3. JUSTIFICACIÓN DEL PROYECTO

3.1 Necesidad del mercado

Los proyectos de casino con GUI nativa y calidad visual cercana a la de una aplicación profesional son escasos en el ámbito académico y de aprendizaje. Casino Royale Premium cubre ese nicho ofreciendo una base sólida, modular y extensible que puede servir como portafolio técnico, herramienta de aprendizaje sobre arquitectura MVC, persistencia SQLite o distribución con PyInstaller.

3.2 Innovación tecnológica

- ◆ Interfaz reactiva con Flet (Flutter en Python) con animaciones nativas
- ◆ Generación de aleatoriedad criptográfica en el barajado con secrets
- ◆ Sistema de autenticación sobre SQLite con rate-limiting y WAL
- ◆ Migración automática desde JSON a SQLite sin pérdida de datos
- ◆ Animación de ruleta con física simulada (desaceleración, bounce, corrección de ángulo)
- ◆ Empaquetado como ejecutable portable mediante PyInstaller + .spec personalizado

3.3 Beneficios esperados

Para el jugador

- Experiencia visual inmersiva con temática de casino de alta gama
- Tres juegos distintos con lógica auténtica y animaciones fluidas
- Saldo y historial persistente entre sesiones
- Interfaz adaptable e intuitiva sin curva de aprendizaje

Para el desarrollador

- Portafolio técnico que demuestra dominio de GUI, POO, BD y distribución
- Código modular y fácilmente extensible con nuevos juegos
- Arquitectura de referencia para proyectos Flet de mediana complejidad
- Demostración de buenas prácticas: hashing SHA-256, separación de capas, WAL

4. TECNOLOGÍAS EMPLEADAS

4.1 Lenguaje de programación

Python 3.x — lenguaje principal de todo el proyecto

4.2 Base de datos

SQLite3 — base de datos relacional embebida, sin servidor. Dos bases de datos separadas: casino_auth.db (autenticación) y casino.db (datos de jugador).

4.3 Librerías principales

Librería	Uso en el proyecto
flet	Interfaz gráfica reactiva multiplataforma (Flutter en Python)
sqlite3	Acceso a BD para autenticación y persistencia de jugadores
hashlib	Hashing SHA-256 de contraseñas y respuestas de seguridad
winsound	Reproducción de efectos de sonido (.wav) en Windows
asyncio	Gestión de animaciones asíncronas sin bloquear la UI
PyInstaller	Empaquetado del proyecto como ejecutable .exe portable
secrets	Generación criptográficamente segura para el barajado
pathlib / json	Gestión de rutas y configuración persistente
dataclasses	Modelos de datos: Jugador, SessionStats
re / datetime	Validaciones de contraseña y marcas temporales del historial

4.4 Arquitectura

- Diseño modular por vistas independientes: cada juego en su propio fichero
- Patrón MVC ligero: modelos en models.py, autenticación en auth_db.py, vistas en casino_pages/
- Dos capas de base de datos desacopladas: autenticación (AuthDB) y datos de juego (PersistenciaCasino)
- Sistema de rate-limiting en la capa de autenticación (5 intentos / 15 min)
- Singleton de página Flet inyectado en cada vista como dependencia
- Migración automática de usuarios desde JSON a SQLite en el primer arranque

5. INGENIERÍA DEL SOFTWARE

5.1 Storyboard (ficheros principales)

Fichero	Responsabilidad
casino_premium.py	Punto de entrada, autenticación, dashboard y navegación principal
casino_pages/blackjack_view.py	Vista y lógica completa del juego Blackjack
casino_pages/roulette_view.py	Vista de Ruleta con animación de rueda y gestión de apuestas
casino_pages/slots_view.py	Vista de Tragaperras con animación de rodillos y paylines
casino_pages/caja_view.py	Panel de caja: depósitos, retiros e historial de movimientos
secciones/auth_db.py	AuthDB: registro, login, recuperación y rate-limiting sobre SQLite
secciones/models.py	Clases Jugador, SessionStats y PersistenciaCasino
secciones/games.py	Lógica de juego por consola (Slots, Ruleta, Blackjack)
secciones/config.py	Carga y mezcla de configuración JSON con valores por defecto
secciones/reportes.py	Exportación de sesión a JSON para análisis externo
data/assets/schema.sql	Esquema SQL: tablas usuarios y login_attempts con vista de rate-limit
casino_premium.spec	Fichero de configuración para empaquetar con PyInstaller

5.2 Diagrama de flujo (lógico)

Usuario	Autenticación → Dashboard (vista inicio)
Dashboard	Menú lateral → Caja / Slots / Ruleta / Blackjack / Cerrar sesión
Login	Verificar usuario → Consulta SQLite → Rate-limit → Acceso o error
Registro	Validar datos → Encriptar SHA-256 → INSERT en SQLite
Recuperación	Pregunta seguridad → Verificar respuesta → Resetear contraseña
Caja	Depositar / Retirar → Actualizar saldo → Guardar en BD → Historial
Slots	Colocar apuesta → Animar rodillos → Evaluar paylines → Actualizar saldo
Ruleta	Seleccionar fichas → Apostar en tablero → Animar rueda+bola → Resolver
Blackjack	Apostar → Repartir → Pedir/Plantar → Jugar dealer → Pagar
Fin de jugada	Registrar en historial → guardar_jugador() → refresh_balance()

5.3 Estructura del proyecto

```
casino_royale_premium/ ■■■■ casino_premium.py ■■■■ casino_premium.spec ■■■■ casino_pages/
■ ■■■■ __init__.py ■ ■■■■ blackjack_view.py ■ ■■■■ roulette_view.py ■ ■■■■ slots_view.py
■ ■■■■ caja_view.py ■■■■ secciones/ ■ ■■■■ __init__.py ■ ■■■■ auth_db.py ■ ■■■■
models.py ■ ■■■■ games.py ■ ■■■■ config.py ■ ■■■■ database.py ■ ■■■■ reportes.py ■■■■
data/ ■ ■■■■ casino.db ■ ■■■■ casino_data.json ■ ■■■■ assets/ ■ ■■■■ casino_auth.db ■
■■■■ schema.sql ■■■■ assets/ ■ ■■■■ cards/ (imágenes de cartas PNG) ■ ■■■■ slots/
(símbolos de tragaperras PNG) ■ ■■■■ roulette/ (rueda PNG) ■ ■■■■ sounds/ (action.wav,
win.wav, tick.wav)
```

6. DIFICULTADES ENCONTRADAS

6.1 Técnicas

- Sincronización de la animación de la bola de ruleta con la posición real del número resultado: requirió calibración trigonométrica y ajuste fino del offset
- Gestión de async/await en Flet para animaciones sin bloquear el hilo principal de la UI
- Construcción del tablero de ruleta con áreas clicables de tamaños variables y overlay de fichas por casilla
- Renderizado correcto de imágenes PNG (cartas, símbolos) en Flet con rutas relativas y fit de contenedor
- Empaquetado con PyInstaller incluyendo todos los assets (imágenes, sonidos, base de datos)

6.2 Base de datos

- Separación en dos bases de datos: auth_db para credenciales y casino.db para datos de juego, asegurando desacoplamiento y seguridad
- Implementación del rate-limiting mediante vista SQL v_intentos_recientes con ventana temporal de 15 minutos
- Migración automática idempotente desde casino_auth.json a SQLite sin pérdida de datos de usuarios existentes
- Gestión transaccional con PRAGMA WAL para evitar bloqueos en escrituras concurrentes
- Sincronización del saldo en tiempo real: cada jugada actualiza BD y refresca la barra lateral

6.3 Lógica de negocio

- Cálculo del valor de mano en Blackjack con ases dinámicos (11 ó 1 según conveniencia)
- Detección precisa del ángulo de la bola en la ruleta respecto a la rueda girada para mapear el número ganador
- Evaluación de las 5 paylines de la tragaperras (3 horizontales + 2 diagonales) con acumulación de premios múltiples
- Validación de contraseña con requisitos de seguridad (longitud, número, símbolo especial)
- Control del flujo de juego en Blackjack: seguro, split, doble y variante soft-17 del dealer

7. ANÁLISIS DAFO

7.1 Matriz DAFO

FORTALEZAS • Interfaz visual premium (paleta oscura, dorada, animaciones) • Tres juegos con lógica auténtica y efectos de sonido • Arquitectura modular y fácilmente extensible • Autenticación segura con rate-limiting sobre SQLite • Distribución portable como ejecutable único	DEBILIDADES • Efectos de sonido con winsound (solo Windows) • Sin soporte multijugador ni red • SQLite limitado para volúmenes muy grandes • Sin sistema de logros visible en la UI actual
OPORTUNIDADES • Migración a interfaz web con FastAPI + React • Modo multijugador en red local • Sistema de torneos y clasificaciones • Nuevos juegos: Póker, Baccarat, Video Póker • App móvil complementaria con Flet mobile	AMENAZAS • Plataformas macOS/Linux sin soporte winsound • Cambios en la API de Flet (librería en desarrollo activo) • Regulaciones legales sobre simuladores de juego • Competencia de casinos online con mayor contenido

8. ANÁLISIS DE DATOS

8.1 Tablas principales

Tabla	Descripción	Columnas clave
usuarios (casino_auth.db)	Credenciales y preguntas de seguridad	id, username, password_hash, security_question, security_answer_hash, last_login
login_attempts (casino_auth.db)	Registro de intentos de login para rate-limiting	id, username, success, attempt_time
v_intentos_recientes (VIEW)	Vista que agrega intentos fallidos en ventana de 15 min	username, intentos_fallidos
users (casino.db)	Datos de juego, saldo e historial de cada jugador	username, password_hash, saldo, historial (JSON), estadisticas_globales (JSON)

8.2 Estructura del historial (JSON)

Cada entrada del historial de un jugador sigue el siguiente esquema JSON, almacenado en la columna historial de la tabla users (máximo 200 entradas):

```
{
  "fecha": "2026-03-09 18:45:22",
  "juego": "Ruleta",
  "apuesta": 50.0,
  "premio": 100.0,
  "balance": 50.0,
  "detalle": "Cayó 19",
  "saldo_post": 1150.0
}
```

9. PROYECCIÓN FUTURA

9.1 Mejoras inmediatas

- Compatibilidad multiplataforma del audio: reemplazar winsound por pygame.mixer o playsound
- Sistema de logros y gamificación visible en la UI (nivel, XP, badges desbloqueados)
- Estadísticas de sesión en pantalla: RTP real, racha de victorias, mayor ganancia
- Panel de ajustes: tema visual claro/oscuro, sonido on/off, límite de apuesta configurable
- Soporte para múltiples perfiles de jugador en una misma instalación
- Exportación del historial a PDF o CSV desde la propia app

9.2 Funcionalidades avanzadas

- Migración a interfaz web con FastAPI (backend REST) y React o Flet web (frontend)
- Modo multijugador en red local: partidas de Blackjack y Póker entre usuarios
- Sistema de torneos con tabla de clasificación compartida y premios en saldo virtual
- Integración de nuevos juegos: Video Póker, Baccarat, Craps y Ruleta americana
- Aplicación móvil complementaria con Flet Mobile para iOS y Android
- Inteligencia Artificial para sugerencias de estrategia básica en Blackjack
- Panel de administración web para supervisar estadísticas y gestionar usuarios

10. CONCLUSIONES

10.1 Logros alcanzados

- ✓ Aplicación de escritorio completamente funcional con interfaz gráfica profesional y temática de casino
- ✓ Sistema de autenticación seguro con SHA-256, rate-limiting y recuperación por pregunta de seguridad
- ✓ Tres juegos completos con lógica auténtica, animaciones fluidas y efectos de sonido
- ✓ Arquitectura modular y limpia con separación total entre autenticación, lógica de juego y presentación
- ✓ Persistencia de datos robusta en SQLite con migraciones automáticas desde JSON
- ✓ Distribución portable como ejecutable único sin dependencias externas
- ✓ Animación de ruleta con física simulada (velocidad, bounce, corrección de ángulo)
- ✓ Sistema de caja completo con historial detallado y actualización en tiempo real del saldo
- ✓ Tragaperras con 5 paylines, sistema de pesos y acumulación de premios múltiples
- ✓ Clases POO (Jugador, SessionStats, Juego) con encapsulamiento, herencia y serialización

10.2 Valoración personal

El desarrollo de Casino Royale Premium ha representado un reto técnico exigente y muy enriquecedor. Ha permitido integrar conocimientos de programación orientada a objetos en Python, diseño de interfaces reactivas con Flet, gestión de bases de datos relacionales normalizadas, animación procedural y empaquetado de aplicaciones de escritorio. El resultado es una aplicación visualmente pulida, funcionalmente completa y técnicamente sólida que puede servir como base para proyectos más ambiciosos o como portafolio profesional.

10.3 Impacto esperado

- Demostración práctica de dominio técnico en desarrollo de aplicaciones de escritorio con Python
- Base de código extensible para añadir nuevos juegos o migrar a una plataforma web
- Referencia de arquitectura para proyectos Flet de mediana-alta complejidad
- Experiencia de usuario cercana a una aplicación comercial, destacable en portafolio académico

Fecha: 2026 · Versión del documento: 1.0 · Autor: Luis Villegas Rivera